

ICSS Compiler – Eigen Taaluitbreidingen

1. Inleiding

Als uitbreiding op de standaard ICSS-functionaliteit zijn meerdere semantische regels en optimalisaties geïmplementeerd.

De volgende uitbreidingen zijn toegevoegd:

1. **Type-consistente variabelen**
2. **AST-optimalisaties (opschonen van de boom)**
3. **Striktere validatie van operaties (extra foutafhandeling)**

Deze uitbreidingen verhogen de betrouwbaarheid, voorspelbaarheid en kwaliteit van de compiler.

2. Overzicht van uitbreidingen

Uitbreiding	Type
Variabelen met vast type	Semantisch
AST-optimalisatie (clean-up)	Optimalisatie
Striktere operatie-validatie	Semantisch

3. Uitbreiding 1 – Type-consistente variabelen

Probleem

In standaard ICSS kunnen variabelen meerdere keren worden overschreven met verschillende types:

```
A := 10px;
```

```
A := 20%;
```

Dit kan leiden tot:

- inconsistent gedrag
- fouten die pas laat zichtbaar worden
- moeilijk te debuggen code

Oplossing

Variabelen mogen binnen dezelfde scope niet van type veranderen.

Implementatie

Geïmplementeerd in de Checker:

```
ExpressionType newType = resolveType(assignment.expression);

String name = assignment.name.name;
ExpressionType existing = variableScopes.getFirst().get(name);

if (existing != null && existing != newType) {
    assignment.setError("Variable type cannot change");
}

variableScopes.getFirst().put(name, newType);
```

Scope-gedrag

- Zelfde scope $\rightarrow$ type mag niet veranderen
- Nieuwe scope $\rightarrow$ toegestaan

```
A := 10px;

h1 {
    A := 20px; // toegestaan
}
```

4. Uitbreiding 2 – AST-optimalisaties

Naast semantische checks zijn optimalisaties toegevoegd aan de AST-transformatie.

4.1 Verwijderen van variabele-assignments

Na evaluatie worden `VariableAssignment` nodes verwijderd uit de AST.

Reden:

- variabelen zijn al vervangen door literals
 - ze zijn niet meer nodig voor codegeneratie
-

4.2 Verwijderen van dubbele declaraties

Wanneer dezelfde property meerdere keren voorkomt:

```
h1 {  
  width: 10px;  
  width: 20px;  
}
```

→ Alleen de laatste declaratie blijft behouden.

```
h1 {  
  width: 20px;  
}
```

Dit volgt het gedrag van CSS (laatste waarde wint).

Meerwaarde

- schonere AST
 - minder redundantie
 - correctere CSS-output
-

5. Uitbreiding 3 – Striktere operatie-validatie

De standaard opdracht vereist typecontrole, maar de implementatie is uitgebreid met extra robuustheid.

Extra checks:

- kleuren in operaties verboden
- booleans in operaties verboden
- verschillende types bij + en - verboden
- vermenigvuldiging zonder scalar verboden

```
if (left == ExpressionType.COLOR || right == ExpressionType.COLOR) {  
  op.setError("Cannot use color in operations");  
}
```

Waarom dit een uitbreiding is

Hoewel typechecks deels gevraagd zijn, is deze implementatie:

- uitgebreider dan minimum
 - consistenter toegepast
 - robuuster tegen edge cases
-

6. Toegevoegde testcases

Extra testbestanden:

- `level4.icss` → complexere expressies
 - `level5.icss` → nested if/else
 - `CheckerTest.java` → Automated tests for semantic validation (undefined variables, invalid operations, type checking)
 - `EvaluatorTest.java` → Automated tests for expression evaluation and arithmetic operations
-

7. Meerwaarde van de uitbreidingen

De uitbreidingen zorgen voor:

- vroegtijdige foutdetectie
 - consistent typegebruik
 - betere onderhoudbaarheid van ICSS-code
 - schonere en correctere CSS-output
 - realistischer gedrag vergelijkbaar met echte compilers
-

8. Verwachte punten

De uitbreidingen vallen onder:

- “Iedere variabele mag alleen een vast type hebben”
- “Het implementeren van optimalisaties op de AST”

Geschatte waarde: **15–20 punten**

9. Gebruik van AI-hulpmiddelen

Bij het opstellen van dit document is gebruikgemaakt van AI-ondersteuning voor het structureren en formuleren van de tekst.

De inhoud, keuzes, implementatie en technische uitwerking van de opdracht zijn zelfstandig uitgevoerd en begrepen door de auteur.

10. Conclusie

Door meerdere uitbreidingen te combineren is de compiler niet alleen functioneel correct, maar ook robuuster en efficiënter geworden.

De combinatie van strengere semantiek en AST-optimalisatie brengt de implementatie dichterbij hoe echte compilers werken.

Hiermee overstijgt de oplossing de basisvereisten van de opdracht.